HUFE 数据结构第二讲

2026 年 3 月 21 日

1 本次课定位

本次课对应 5 次课安排中的第 2 次课，时长 3 小时。本讲不再包含“数组和广义表”，而是把时间集中到**树和二叉树**，重点突破二叉树性质、遍历、线索二叉树与哈夫曼树。

这一讲比第一讲明显更难，因为题目不再只考“定义记忆”，而是开始要求同学把**结构性质、遍历规则和应用题计算**连起来理解。

1.1 本次课目标

- 理解树、森林、结点的度、层次、高度、叶子结点等基本概念。
- 掌握二叉树的定义与常见特殊形态，尤其是满二叉树与完全二叉树。
- 熟练掌握二叉树的几个核心性质，并能用于选择题和计算题。
- 掌握二叉树的顺序存储、链式存储以及常见下标关系。
- 掌握先序、中序、后序、层序遍历的访问顺序及典型逆向判断题。
- 了解线索二叉树的基本思想，能区分“孩子指针”和“线索指针”。
- 掌握哈夫曼树构造过程和带权路径长度 WPL 的计算。

1.2 考试导向提醒

这一讲在试卷中属于**高频 + 中高难**内容，常见出题方式主要有四类：

- **概念判断题**：如树与二叉树的区别、完全二叉树的定义、线索化后的指针含义。
- **性质计算题**：如由结点层数求最大结点数，由叶子数推度为 2 的结点数。
- **遍历分析题**：给定一棵树写遍历序列，或由两种遍历结果反推出树形。
- **应用构造题**：如哈夫曼树构造、WPL 计算、层序编号关系判断。

所以本讲复习不能只记“名词解释”，而要建立一条清晰主线：**概念 - 性质 - 存储 - 遍历 - 应用**。

1.3 3 小时建议节奏

阶段	时间	主要内容
第 1 段	35 分钟	树的基本概念、术语、树与森林、树的常见表示思想
第 2 段	55 分钟	二叉树定义、特殊二叉树、核心性质、顺序与链式存储
第 3 段	55 分钟	先序、中序、后序、层序遍历，遍历序列分析题
第 4 段	35 分钟	线索二叉树、哈夫曼树、WPL 计算、课堂总结

© 2026 isomoes

2 树的基础概念

2.1 树的定义

树是 n 个结点的有限集合。当 $n=0$ 时，称为空树；当 $n>0$ 时，有且仅有一个特定结点称为**根结点**，其余结点可分成若干个互不相交的子集，每个子集本身又是一棵树，称为原树的**子树**。

这个定义里最重要的不是句子长，而是抓住三点：

- 非空树有且仅有一个根。
- 除根外，其余结点都有且仅有一个直接前驱。
- 一个结点可以有多个直接后继，因此树是典型的一对多结构。

2.2 树中的常见术语

- **结点的度**：该结点拥有的子树个数。
- **树的度**：整棵树中各结点度的最大值。
- **叶子结点**：度为 0 的结点，也叫终端结点。
- **分支结点**：度不为 0 的结点，也叫非终端结点。
- **孩子、双亲、兄弟**：由直接连接关系确定的相对关系。
- **路径**：从一个结点到另一个结点所经过的结点序列。
- **结点的层次**：从根开始定义，根通常记为第 1 层。
- **树的高度**：树中结点的最大层次，也叫树的深度。

2.3 树与线性结构的区别

第一讲中的线性表是一对一关系，而树是一对多关系，因此分析方法也会变化：

- 线性结构更强调前驱和后继唯一。
- 树结构更强调层次、分支和递归定义。
- 在线性结构中常见的是“从前往后”；在树结构中常见的是“从根到子树”。

这也是为什么学树时，**递归思想**会变得很自然：因为“每棵子树仍然是树”。

© 2026 isomoes

2.4 森林的概念

森林是若干棵互不相交的树的集合。
考试中容易出现的理解是:

- 一棵树去掉根结点后，可能变成一个森林。
- 给森林增加一个统一的根结点，可以转化为一棵树。

因此树和森林之间可以相互转化，只是观察角度不同。

2.5 树的常见表示思想

普通树不像二叉树那样天然只有两个孩子，所以表示方式更多是“了解性考点”:

1. **双亲表示法**：每个结点记录其双亲位置，便于找父结点。
2. **孩子表示法**：每个结点记录其所有孩子，便于找子结点。
3. **孩子兄弟表示法**：每个结点用两个指针分别指向第一个孩子和下一个兄弟，可把普通树转成二叉链式形式。

其中最值得记的是第三种，因为它为树和二叉树之间的联系提供了桥梁。

2.6 树的基础练习

1. 树为什么属于非线性结构?
2. 结点的度和树的度分别是什么意思?
3. 叶子结点和分支结点如何区分?
4. 一棵非空树中，除根结点外的其他结点各有什么共同特征?
5. 请判断：森林中每个元素都必须是二叉树。(　　)

3 二叉树

3.1 定义

二叉树是另一类重要的树形结构。它的每个结点最多只有两棵子树，并且这两棵子树有左右之分，分别称为**左子树**和**右子树**。
这里要特别强调：二叉树不是简单的“度不超过 2 的树”，它还是**有序树**。
也就是说:

- 左子树和右子树不能随意交换。
- 即使某结点只有一个孩子，也要区分它是左孩子还是右孩子。

© 2026 isomoes

3.2 二叉树与普通树的区别

1. 二叉树每个结点最多两个孩子；普通树不一定。
2. 二叉树左右子树有顺序；普通树的孩子次序未必是核心属性。
3. 二叉树即使某层为空，也可能影响后续结构判断，尤其在完全二叉树中最明显。

3.3 特殊二叉树

3.3.1 满二叉树

满二叉树指一棵高度为 h 的二叉树，其每一层的结点数都达到最大值。此时总结点数为：

$$2^h - 1$$

它的特点是“长得最规整”，每个分支结点都有两个孩子，所有叶子都在同一层。

3.3.2 完全二叉树

完全二叉树指：除最后一层外，其余各层都满；最后一层的结点连续集中在最左边。
完全二叉树是考试重点，因为它和顺序存储联系最紧密。
常见判断角度：

- 若最后一层有空位，只能出现在最右侧。
- 一旦某位置空了，它右边不能再出现结点。
- 完全二叉树不一定是满二叉树，但满二叉树一定是完全二叉树。

3.3.3 斜树

如果一棵二叉树所有结点都只有左孩子，称为左斜树；都只有右孩子，称为右斜树。

这种结构在遍历题中很容易判断，但在性质题中要注意：它虽然仍是二叉树，却已经很接近线性结构。

3.4 二叉树的核心性质

3.4.1 性质 1：第 i 层最多有多少个结点

二叉树第 i 层上至多有：

$$2^{i-1} \quad (i \geq 1)$$

理解方法很简单：根第 1 层最多 1 个，第 2 层最多 2 个，第 3 层最多 4 个，每下一层都最多翻倍。

© 2026 isomoes

3.4.2 性质 2：高度为 h 的二叉树最多有多少个结点

若二叉树高度为 h，则最多有：

$$2^h - 1$$

这是把每一层最多结点数相加得到的结果：

$$1 + 2 + 4 + \cdots + 2^{h-1} = 2^h - 1$$

3.4.3 性质 3：度为 0 和度为 2 的结点关系

对任意一棵非空二叉树，若叶子结点个数为 n_0，度为 2 的结点个数为 n_2，则有：

$$n_0 = n_2 + 1$$

这个性质几乎是必考性质之一。注意它只和**叶子结点数、度为 2 的结点数**直接相关，与度为 1 的结点数无直接等式关系。

3.4.4 性质 4：完全二叉树编号关系

若完全二叉树按层序从 1 开始编号，则编号为 i 的结点：

- 双亲编号为 $\lfloor i/2 \rfloor$（当 $i > 1$）
- 左孩子编号为 $2i$
- 右孩子编号为 $2i + 1$

这个关系是顺序存储判断题的核心基础。

3.5 性质理解例子

1. 若某二叉树第 4 层，则该层最多有 $2^3 = 8$ 个结点。
2. 若某二叉树高度为 5，则最多有 $2^5 - 1 = 31$ 个结点。
3. 若某二叉树有 10 个叶子结点，则度为 2 的结点数一定为 9。
4. 在完全二叉树中，若某结点编号为 6，则其双亲编号为 3，左孩子编号为 12，右孩子编号为 13。

3.6 二叉树的存储结构

3.6.1 顺序存储

顺序存储通常适用于**完全二叉树**，因为它层次紧凑，不会浪费太多空间。
其核心思想是：按层序把结点依次放入数组。
优点：

- 结点之间的位置关系可直接通过下标计算。
- 访问编号结点方便。

缺点:

- 若不是完全二叉树，会产生较多空位，空间利用率下降。

3.6.2 链式存储

链式存储更适合一般二叉树，通常使用二叉链表:

```
typedef struct BiNode {
    ElemType data;
    struct BiNode *lchild;
    struct BiNode *rchild;
} BiNode, *BiTree;
```

这里:

- `data` 存放结点值;
- `lchild` 指向左孩子;
- `rchild` 指向右孩子。

若某个孩子不存在，对应指针域为空。这些“空指针域”也是后面线索二叉树能够出现的基础。

3.7 二叉树部分常见判断

1. 二叉树并不等价于度不超过 2 的有序树，这个说法虽然接近，但答题时仍应强调“左右子树有次序”。
2. 满二叉树一定是完全二叉树；完全二叉树不一定是满二叉树。
3. 只有完全二叉树的顺序存储下标关系才最自然、最常考。
4. 二叉树中某结点只有一个孩子时，要区分它到底是左孩子还是右孩子。

3.8 二叉树练习

1. 二叉树为什么说是有序树?
2. 满二叉树和完全二叉树有什么关系?
3. 二叉树第 i 层最多有多少个结点?
4. 若某非空二叉树叶子结点数为 7，则度为 2 的结点数是多少?
5. 在完全二叉树中，若结点编号为 9，则其双亲编号是多少?
6. 请判断：完全二叉树的最后一层结点可以任意分散出现。(　　)

4 二叉树的遍历

4.1 为什么遍历是重点

遍历就是按某种规则，把树中所有结点不重不漏地访问一次。
考试中之所以高频考遍历，是因为它同时考了三件事：

- 你是否真正理解“根、左、右”的递归结构；
- 你是否能区分不同遍历顺序；
- 你是否能由遍历序列反推树形或分析应用场景。

4.2 先序遍历

先序遍历的规则是：

$$\text{根} \to \text{左子树} \to \text{右子树}$$

记忆口诀：**先看根**。

```
void PreOrder(BiTree T) {
    if (T != NULL) {
        visit(T);
        PreOrder(T->lchild);
        PreOrder(T->rchild);
    }
}
```

先序遍历常用于“先处理当前结点，再处理子问题”的场景。

4.3 中序遍历

中序遍历的规则是：

$$\text{左子树} \to \text{根} \to \text{右子树}$$

记忆口诀：**根在中间**。

```
void InOrder(BiTree T) {
    if (T != NULL) {
        InOrder(T->lchild);
        visit(T);
        InOrder(T->rchild);
    }
}
```

中序遍历在二叉排序树中尤其重要，因为它会得到一个递增序列。

© 2026 isomoes

4.4 后序遍历

后序遍历的规则是:

$$\text{左子树} \to \text{右子树} \to \text{根}$$

记忆口诀:**最后看根**。

```
void PostOrder(BiTree T) {
    if (T != NULL) {
        PostOrder(T->lchild);
        PostOrder(T->rchild);
        visit(T);
    }
}
```

后序遍历常用于“先处理左右子问题,再处理当前结点”的场景,例如求树高、释放结点空间等。

4.5 层序遍历

层序遍历的规则是:按照从上到下、从左到右的顺序逐层访问。

它和前三种递归遍历不同,通常借助**队列**实现,因此也和第一讲内容形成了连接。

```
void LevelOrder(BiTree T) {
    InitQueue(Q);
    if (T != NULL)
        EnQueue(Q, T);
    while (!QueueEmpty(Q)) {
        DeQueue(Q, p);
        visit(p);
        if (p->lchild != NULL)
            EnQueue(Q, p->lchild);
        if (p->rchild != NULL)
            EnQueue(Q, p->rchild);
    }
}
```

4.6 四种遍历的对比

遍历方式	访问顺序核心	记忆点
先序	根左右	先看根
中序	左根右	根在中间
后序	左右根	最后看根
层序	自上而下逐层	借助队列

© 2026 isomoes

4.7 遍历序列分析例子

设某二叉树结构为：根结点为 A，左子树根为 B，右子树根为 C；其中 B 的左右孩子分别为 D、E，C 只有右孩子 F。

则有：

- 先序遍历：A, B, D, E, C, F
- 中序遍历：D, B, E, A, C, F
- 后序遍历：D, E, B, F, C, A
- 层序遍历：A, B, C, D, E, F

这组序列非常适合反复练习，因为它包含了“左子树完整、右子树不完整”的典型情况。

4.8 由遍历反推树形时要注意什么

1. 仅知道一种遍历序列，通常不能唯一确定一棵二叉树。
2. 若已知**先序 + 中序**，通常可以唯一确定一棵二叉树。
3. 若已知**后序 + 中序**，通常也可以唯一确定一棵二叉树。
4. 若只有先序和后序，在一般情况下不能唯一确定。

这里的核心原因是：中序遍历把“根左边”和“根右边”分得很清楚，因此能提供结构划分依据。

4.9 遍历练习

1. 先序、中序、后序遍历分别把根结点放在什么位置访问?
2. 层序遍历为什么通常要借助队列?
3. 为什么中序遍历在反推二叉树时特别关键?
4. 若某二叉树先序为 A, B, D, E, C，中序为 D, B, E, A, C，则根结点是谁?
5. 请判断：只知道一棵二叉树的先序遍历序列，就一定能唯一确定该树。（　　）

5 线索二叉树

5.1 为什么需要线索二叉树

普通二叉链表中，很多结点的左指针或右指针可能为空。若这些空指针不利用，就会造成一定浪费。

线索二叉树的基本思想就是：把这些空指针改造为指向某种遍历次序下的**前驱**或**后继**。

因此它的目标不是“改变树”，而是让遍历和查找前驱后继更方便。

5.2 基本概念

若某结点原来的左指针为空，可让它指向在某种遍历序列中的前驱；若右指针为空，可让它指向该结点在该遍历序列中的后继。这样的指针就称为**线索**。

按照建立线索所依据的遍历方式不同，可分为：

- 前序线索二叉树
- 中序线索二叉树
- 后序线索二叉树

其中考试中最常见的是**中序线索二叉树**。

5.3 存储结构中的标志位

为了区分一个指针到底是“孩子指针”还是“线索指针”，通常增加两个标志位：

```
typedef struct ThreadNode {
    ElemType data;
    struct ThreadNode *lchild, *rchild;
    int ltag, rtag;
} ThreadNode, *ThreadTree;
```

常见约定：

- `ltag = 0` 表示 `lchild` 指向左孩子；`ltag = 1` 表示指向前驱线索。
- `rtag = 0` 表示 `rchild` 指向右孩子；`rtag = 1` 表示指向后继线索。

这类题目的关键不是背代码，而是分清“空指针被改造成线索以后，必须有标志位说明其意义”。

5.4 线索二叉树易错点

1. 线索不是额外新加一条边，而是对原空指针域的利用。
2. 建线索所依据的遍历不同，前驱和后继关系也会不同。
3. 线索化后，不是所有指针都变成线索，原本存在孩子的指针仍然是孩子指针。

5.5 线索二叉树练习

1. 线索二叉树为什么能够提高遍历效率？
2. 为什么线索二叉树需要增加标志位？
3. 中序线索二叉树中的线索关系是依据哪一种遍历建立的？
4. 请判断：线索二叉树中所有左右指针都一定改成线索。（　　）

6 哈夫曼树

6.1 哈夫曼树的核心思想

哈夫曼树也叫**最优二叉树**。它研究的不是“形状是否好看”，而是：在带权叶子结点已知时，如何使带权路径长度最小。

它最常考的两个点是：

- 哈夫曼树如何构造；
- 带权路径长度 WPL 如何计算。

6.2 路径长度与带权路径长度

- **路径长度**：从根到该结点所经过的边数。
- **带权路径长度**：结点权值乘以该结点路径长度。
- **WPL**：所有叶子结点带权路径长度之和。

若叶子结点权值分别为 $w_1, w_2, \ldots, w_n$，对应路径长度为 $l_1, l_2, \ldots, l_n$，则：

$$WPL = \sum_{i=1}^{n} w_i l_i$$

6.3 哈夫曼树构造规则

构造过程可概括为：

1. 把每个权值看作一棵只有一个结点的树。
2. 每次选出权值最小的两棵树。
3. 把它们合并成一棵新树，新根权值等于两者之和。
4. 重复上述过程，直到森林中只剩一棵树。

最关键的规则就一句话：**每次取最小的两个合并**。

6.4 哈夫曼树计算示例

设权值集合为：

$$\{2, 4, 5, 7\}$$

构造步骤：

1. 先取最小的 2 和 4，合并成 6。

© 2026 isomoes

2. 当前集合变为 $\{5, 6, 7\}$，再取 5 和 6，合并成 11。

3. 当前集合变为 $\{7, 11\}$，再合并成 18。

若最终树中各叶子的路径长度分别为:

- 权值 7 的路径长度为 1
- 权值 5 的路径长度为 2
- 权值 2 的路径长度为 3
- 权值 4 的路径长度为 3

则:

$$WPL = 7 \times 1 + 5 \times 2 + 2 \times 3 + 4 \times 3 = 35$$

6.5 哈夫曼树的几个结论

1. 权值越大的叶子，通常离根越近，编码也越短。
2. 哈夫曼树中没有度为 1 的结点。
3. 哈夫曼树不一定唯一，但最小 WPL 相同。

6.6 哈夫曼编码的理解

哈夫曼编码就是在哈夫曼树基础上形成的前缀编码。通常约定:

- 向左走记为 0
- 向右走记为 1

这样从根走到某叶子形成的一串 0/1，就是该字符的编码。
它的优势在于：高频字符编码短，低频字符编码长，总体压缩效率更高。

6.7 哈夫曼树练习

1. 哈夫曼树为什么又叫最优二叉树?
2. WPL 的计算对象是所有结点还是所有叶子结点?
3. 构造哈夫曼树时，每一步为什么都要选权值最小的两棵树?
4. 请判断：哈夫曼树中可能存在只有一个孩子的结点。(　　)

© 2026 isomoes

7 易混点总结

1. **树与二叉树**：二叉树不是普通树的特例描述那么简单，它强调左右次序。
2. **满二叉树与完全二叉树**：满二叉树更严格，完全二叉树更常考。
3. **第 i 层最多结点数与高度为 h 的最多结点数**：一个看单层，一个看整树。
4. **先序、中序、后序**：关键不是背名字，而是根结点出现在访问序列中的位置。
5. **层序遍历与前面三种遍历**：层序通常借助队列，不完全依赖递归。
6. **线索与孩子**：线索二叉树中的指针不一定都指向孩子，需要靠标志位区分。
7. **哈夫曼树与普通二叉树**：哈夫曼树关注的是 WPL 最小，而不是结点值大小规律。

8 本讲知识框架

建议把本讲内容整理成下面这条主线:

主题	核心问题	典型内容	高频考点
树基础	什么是层次结构	根、孩子、度、森林	术语辨析
二叉树	为什么左右有序	满二叉树、完全二叉树	性质计算、编号关系
遍历	如何不重不漏访问	先中后层序	序列判断、反推结构
线索化	空指针如何利用	前驱、后继、标志位	线索含义辨析
哈夫曼树	怎样让 WPL 最小	构造规则、编码	合并步骤、WPL

9 典型例题

9.1 例题 1：二叉树第 i 层结点数

题目： 某二叉树第 5 层最多有多少个结点?
分析： 利用性质“第 i 层最多有 2^{i-1} 个结点”。
答案： $2^4 = 16$。

9.2 例题 2：叶子结点与度为 2 结点关系

题目： 某非空二叉树中叶子结点数为 12，则度为 2 的结点数是多少?
分析： 由性质 $n_0 = n_2 + 1$，得 $n_2 = 11$。
答案： 11。

9.3 例题 3：完全二叉树编号

题目： 在一棵按层序从 1 开始编号的完全二叉树中，编号为 10 的结点的双亲编号是多少?
分析： 双亲编号为 $\lfloor 10/2 \rfloor$。
答案： 5。

© 2026 isomoes

9.4 例题 4：遍历序列判断

题目： 某二叉树先序遍历为 A, B, D, E, C, F，中序遍历为 D, B, E, A, C, F，则根结点是谁?
分析： 先序遍历第一个访问的一定是根结点。
答案： A。

9.5 例题 5：层序遍历使用什么结构

题目： 二叉树层序遍历通常借助什么线性结构实现?
分析： 层序遍历要求先访问先进入下一层的结点，符合先进先出。
答案： 队列。

9.6 例题 6：哈夫曼树的 WPL

题目： 一棵哈夫曼树的叶子权值和路径长度分别为 $(3, 2)$、$(5, 2)$、$(7, 1)$，求其 WPL。
解：

$$WPL = 3 \times 2 + 5 \times 2 + 7 \times 1 = 23$$

答案： 23。

10 课堂练习

10.1 基础练习

1. 树为什么可以用递归思想来理解?
2. 二叉树和普通树相比，最关键的差异是什么?
3. 满二叉树与完全二叉树分别如何定义?
4. 顺序存储为什么更适合完全二叉树?
5. 先序、中序、后序遍历的根结点分别在什么时机访问?

10.2 判断与选择

1. 二叉树中某结点的左右子树可以随意交换而不影响结构定义。（　　）
2. 满二叉树一定是完全二叉树。（　　）
3. 完全二叉树适合采用顺序存储结构。（　　）
4. 只知道一棵二叉树的中序遍历序列，就一定能唯一确定该树。（　　）
5. 线索二叉树中的空指针域可能被用来存放前驱或后继信息。（　　）
6. 哈夫曼树的目标是让所有叶子尽量处在同一层。（　　）

© 2026 isomoes

10.3 计算与分析

1. 某二叉树第 6 层最多有多少个结点?
2. 若某二叉树高度为 4，则最多有多少个结点?
3. 某非空二叉树有 15 个叶子结点，则度为 2 的结点数是多少?
4. 在完全二叉树中，若某结点编号为 14，请写出其双亲、左孩子、右孩子的编号。
5. 设某二叉树先序遍历为 A, B, C, D，后序遍历为 D, C, B, A，为什么一般不能只靠这两种序列唯一确定一棵二叉树?
6. 若哈夫曼树叶子权值分别为 1、3、6，且其路径长度分别为 2、2、1，求 WPL。

10.4 综合小题

题目：某系统需要完成两个任务：

- 任务 A：按层次从上到下输出一棵二叉树的结点；
- 任务 B：根据若干字符出现频率生成压缩编码方案。

请分别指出任务 A 和任务 B 对应的核心数据结构思想，并说明理由。

11 课后小测参考答案

1. 因为树中每棵子树本身仍然是树，整体与局部结构相同。
2. 二叉树强调每个结点最多两个孩子，且左右子树有顺序。
3. 满二叉树每层都达到最大结点数；完全二叉树除最后一层外都满，最后一层结点连续集中在最左边。
4. 因为完全二叉树结构紧凑，按层序编号后可直接通过数组下标反映双亲和孩子关系。
5. 先序先访问根，中序中间访问根，后序最后访问根。
6. 第 1 题错；第 2 题对；第 3 题对；第 4 题错；第 5 题对；第 6 题错。
7. 第 6 层最多有 $2^5 = 32$ 个结点。
8. 高度为 4 时，最多有 $2^4 - 1 = 15$ 个结点。
9. 由 $n_0 = n_2 + 1$ 得度为 2 的结点数为 14。
10. 双亲为 7，左孩子为 28，右孩子为 29。
11. 因为没有中序序列提供左右子树的划分边界，一般无法唯一确定结构。
12. $WPL = 1 \times 2 + 3 \times 2 + 6 \times 1 = 14$。

综合小题参考：任务 A 对应层序遍历，核心借助队列；任务 B 对应哈夫曼树，核心是按频率构造最优二叉树以得到较短编码。

© 2026 isomoes

11.1 分节练习参考答案

树的基础练习参考：

1. 因为一个结点可以对应多个后继，元素之间不是一对一关系。
2. 结点的度是某结点孩子个数；树的度是整棵树中结点度的最大值。
3. 度为 0 的是叶子结点，度不为 0 的是分支结点。
4. 都有且仅有一个直接前驱，也就是双亲结点。
5. 错。森林中的元素是树，不要求每棵树都是二叉树。

二叉树练习参考：

1. 因为左右子树有明确区分，不能随意交换。
2. 满二叉树一定是完全二叉树，但完全二叉树不一定是满二叉树。
3. 第 i 层最多有 2^{i-1} 个结点。
4. 6。
5. 双亲编号为 $\lfloor 9/2 \rfloor = 4$。
6. 错。最后一层必须连续集中在最左边。

遍历练习参考：

1. 先序先访问根；中序根在中间；后序最后访问根。
2. 因为层序遍历要求先访问先进入队列的上一层结点，符合先进先出。
3. 因为中序序列可以把根左边和根右边的结点范围划分出来。
4. 根结点是 A。
5. 错。只知道先序遍历通常不能唯一确定一棵二叉树。

线索二叉树练习参考：

1. 因为空指针可被利用来直接指向某种遍历次序下的前驱或后继。
2. 因为同一个指针域可能表示孩子，也可能表示线索，必须区分。
3. 依据中序遍历建立。
4. 错。原本存在孩子的指针仍然指向孩子。

哈夫曼树练习参考：

© 2026 isomoes

1. 因为它使带权路径长度 WPL 达到最小。
2. 是所有叶子结点。
3. 因为这样才能保证最终带权路径长度最小。
4. 错。哈夫曼树中不存在度为 1 的结点。

12 本讲小结

本讲的核心不只是“认识树”，而是建立树形结构的分析方法：**先分清基本概念，再掌握二叉树性质，然后通过遍历理解递归结构，最后把线索二叉树和哈夫曼树看作具体应用**。

如果说第一讲是在建立数据结构的入门视角，那么这一讲就是正式进入“结构分析题”。其中最需要反复练习的三个点分别是：**完全二叉树编号关系、四种遍历顺序、哈夫曼树的构造与 WPL 计算**。

下一讲将进入图、查找和排序，计算量和综合性会进一步提高，因此这一讲的遍历与结构理解一定要打牢。

© 2026 isomoes